AI TECH CONSULTANT

Assignment Project Document

Prepared for: BusinessLabs – Applied AI Developer / Engineer Capstone Reference date: October 14, 2026 Project duration: 4 days Project type: AI-powered knowledge, email automation, and meeting-to-proposal workflow

1. Executive Summary

This project builds an AI Tech Consultant system that helps a business support team answer customer questions, draft email responses, and turn client meetings into structured scopes of work and proposals.

The system contains three connected tasks:

1. Knowledge Brain: A Python-based knowledge system that collects, indexes, and retrieves information from Brilliant Directories' Solution Center.
2. Inbox Agent: A TypeScript-based email assistant that uses the Knowledge Brain to prepare accurate, evidence-based email replies.
3. Meeting to Scope of Work: A Python-based multi-agent workflow that converts meeting transcripts into requirements, tasks, hour estimates, and a proposal.

The three tasks must work together through APIs. Tasks 2 and 3 must call Task 1's API rather than duplicating its implementation.

The goal is to demonstrate practical AI engineering, reliable tool use, human oversight, evaluation, and clear technical documentation within a four-day development period.

Important: This document describes the proposed implementation and evaluation plan. Results, performance figures, and completed features must only be reported after they have actually been tested.

2. Problem Statement

Business support and consulting workflows often require employees to:

- Search documentation to answer customer questions.
- Write similar email replies repeatedly.
- Ensure that customer answers are accurate and supported by documentation.
- Understand client meeting transcripts and extract requirements.

- Estimate implementation effort.
- Identify requests that are outside the agreed project scope.
- Prepare professional statements of work and proposals.

Performing these tasks manually takes time and may introduce inconsistency, missed requirements, unsupported answers, and inaccurate estimates.

This project proposes an AI-assisted workflow that improves efficiency while keeping humans in control of important decisions.

Project objectives

- Create a reusable, searchable knowledge base.
- Generate grounded answers with source citations.
- Draft customer emails for human review.
- Introduce carefully controlled automation based on demonstrated approval history.
- Convert meeting transcripts into structured project documents.
- Detect requirement changes and potential scope creep.
- Evaluate the system using measurable test cases.
- Produce a maintainable codebase with clear setup and design documentation.

3. Overall System Architecture

The system will contain three main components.

Task 1 — Knowledge Brain

Python · Crawling · Indexing · Retrieval · Cited answers

Task 2 — Inbox Agent

TypeScript

Email classification, grounded drafts, human approval, controlled automation.

Task 3 — Meeting to SOW

Python

Transcript analysis, requirements, estimates, scope changes, proposal generation.

Tasks 2 and 3 consume Task 1 through its API. They must not copy its internal code.

3.1 Technology choices

- Primary AI provider: Claude.
- Task 1: Python.
- Task 2: TypeScript, unless the assignment's permitted Python alternative is needed.
- Task 3: Python.
- Knowledge retrieval: A searchable document store and retrieval pipeline.
- API: A documented HTTP API, with FastAPI as a practical Python option.
- Storage: A lightweight database for documents, metadata, feedback, email decisions, and project records.
- Testing: Automated unit, integration, and evaluation tests.
- Version control: Git with meaningful commits distributed across four days.
- AI coding assistant: Select one permitted tool and use it consistently.

The final implementation should favor reliable, understandable components over unnecessary infrastructure.

5. Task 1 — The Knowledge Brain

5.1 Required workflow

- Read the Brilliant Directories Solution Center sitemap at support.brilliantdirectories.com/support/sitemap.xml.
- Fetch pages in bounded batches with a configurable concurrency limit, delay, timeout, and retry/backoff policy.
- Cache successful results and persist a checkpoint so interrupted ingestion can resume.
- Extract article title, canonical URL, category/folder where available, text, timestamps, image URLs, and image-to-article relationships.
- For screenshots, store image metadata and use a vision-capable model to create a concise, reviewable description. Keep the original image reference and do not treat a generated caption as unquestionable fact.
- Run a daily sitemap comparison. Re-fetch only new or changed pages; mark removed pages inactive so they no longer appear in retrieval results.

- Retrieve relevant chunks, generate an answer only from evidence, and return article citations. If evidence is weak or conflicting, abstain and create a human-expert task.
- Store expert corrections with provenance and review status; make accepted corrections retrievable without silently overwriting source facts.

5.2 Suggested answer contract

Field	Meaning
answer	Plain-language response; may explicitly say that evidence is insufficient.
confidence / status	Calibrated or rule-based signal such as answered, needs_review, or not_found; do not present an uncalibrated score as probability.
sources[]	Article title, canonical URL, relevant snippet or chunk identifier, and last indexed time.
limitations[]	Missing evidence, conflicting sources, stale source, or image interpretation uncertainty.
request_id	Trace identifier for logs and debugging.

5.3 Failure handling

Handle rate limits, network errors, malformed pages, sitemap failures, image-processing failures, and partial ingestion.

Retries must be bounded. Keep the last known good index when a crawl fails; do not interpret a failed crawl as proof that all pages were removed. Track ingestion health and alert the operator to unusually large deletion or update counts.

6. Task 2 — The Inbox Agent

6.1 Email workflow

1. Normalize the incoming email and classify intent: how-to, project request, refund/cancellation, complaint, quote, follow-up, or other.
2. Identify risk and escalation signals such as refund exceptions, legal threats, sensitive data, angry customers, unclear commitments, or unsupported claims.
3. Call Task 1 for product facts and include citations or internal evidence references in the draft record.

4. Generate a structured reply with subject, body, intent, evidence, uncertainty, risk flags, and recommended action.
5. Show the draft to a human who can approve/send, edit, reject, or write a replacement. Store the original draft, final sent version, edit diff, reviewer action, timestamp, and model/prompt version.
6. Use feedback to improve retrieval, templates, examples, or prompts through versioned changes. Evaluate a candidate change before promoting it.

6.2 The 20-consecutive-approval logic and safeguards

The brief allows a category of email to become eligible for autonomous sending after 20 consecutive drafts of the same kind are sent without modification. Treat this as an eligibility gate, not proof of permanent safety.

Control	Recommended rule
Define sameness	Use a stable intent/risk category plus relevant subcategory. Keep high-risk categories permanently human-reviewed unless a separately approved policy permits otherwise.
Count approvals	Count only consecutive, genuinely unedited drafts that were sent. An edit, rejection, correction, or unsafe outcome resets the streak for that category.
Minimum evidence	Persist each of the 20 examples and the policy/model/prompt versions. Require a human to explicitly enable automation for that category.
Guardrails	Never auto-send if retrieval finds no adequate source, sources conflict, the email is out of distribution, or the model output fails schema/safety checks.
Ongoing monitoring	Sample automated replies for review; track corrections, complaints, bounce/escalation rates, and drift.
Revoke autonomy	Immediately pause a category after a serious incident or policy breach. Also pause if rolling quality falls below a documented threshold; require human re-approval to restore.
Auditability	Record why the category was eligible, the approval streak, model version, source evidence, decision, and any later rollback.

The exact rolling threshold should be set before evaluation and documented. Do not invent a performance result to justify enabling autonomous sending.

7. Task 3 — Meeting to Scope

7.1 Three-agent pipeline

Agent / stage	Input and responsibility	Output
1. Requirements Analyst	One or more transcripts; extract goals, actors, workflows, constraints, assumptions, open questions, acceptance criteria, and evidence references.	Structured requirements with transcript name and quote/line/time reference where available.
2. Product Feasibility & Estimator	Check the Knowledge Brain for settings or existing features before assuming custom development. Separate configuration, integration, and custom work; estimate effort with assumptions and uncertainty.	Feasibility findings with citations; task breakdown, hours/ranges, dependencies, risks, and clarification questions.
3. Proposal Writer / Scope Comparator	Assemble approved requirements and estimates into a proposal. For follow-up meetings, compare new evidence against the approved baseline.	Scope of work, estimate, client-ready proposal, and a change report for human review.

7.2 Required documents generated

Scope of Work

- Objectives.
- In-scope features.
- Out-of-scope items.
- Assumptions and dependencies.
- Acceptance criteria.
- Exclusions.

Developer estimate

- Task ID.
- Task description.
- Hours or range.
- Rationale.

- Complexity.
- Dependencies.
- Confidence.
- Risk.

Proposal

- Client problem.
- Proposed solution.
- Deliverables.
- Milestones.
- Estimate summary.
- Assumptions and exclusions.
- Next steps.

Change report

- Unchanged requirements.
- Modified requirements.
- New requests.
- Removed requests.
- Out-of-scope items.
- Estimate impact.
- Questions requiring a decision.

7.3 Change detection and approval

Keep a versioned, human-approved baseline. Compare each follow-up transcript against that baseline, not only against the previous raw transcript.

Every detected change must link to its source reference and state whether it changes cost, schedule, acceptance criteria, or risk.

Never send the revised scope or proposal to the client until a human approves it.

7.4 Transcript requirements

Role-play and record at least three client meetings. One must be a follow-up to the first and must change at least one requirement.

Keep transcripts in the repository with permission to use the recordings; redact unnecessary personal information. Include one ambiguous request so the system demonstrates clarifying questions rather than inventing requirements.

8. Data Models and Traceability

Record	Important fields
Article	source_id, canonical_url, title, content_hash, text/chunks, image_refs, category, last_seen_at, indexed_at, active, crawl_status
Knowledge answer	request_id, question, answer, status, source_ids, uncertainty reasons, model/prompt version, latency, estimated cost
Email case	email_id, intent, risk_level, draft_version, evidence_refs, reviewer_action, edit_diff, sent_text, approval_streak, autonomy_status
Requirement	requirement_id, description, priority, acceptance_criteria, source_transcript, source_quote_or_timestamp, assumptions, status
Estimate item	task_id, requirement_ids, description, estimate_low, estimate_high, rationale, dependencies, risk, estimator_version
Scope change	change_id, baseline_version, new_transcript_ref, change_type, affected_requirements, estimate_delta, approval_status

Every important output should be traceable to source evidence and to the model/prompt version that generated it. Use stable IDs and avoid storing unnecessary customer data.

9. Evaluation Plan

Create at least 20 evaluation cases per task (60 or more total). Use realistic edge cases, not just happy paths. Keep a fixed test set for comparisons and do not tune prompts against every hidden or final evaluation case.

9.1 Evaluation coverage

Task	Suggested case mix (20+ each)	Metrics

Task 1 — Knowledge Brain	Direct answer; multi-article answer; screenshot-dependent answer; stale article; removed article; conflicting sources; no answer; malformed page; rate limit; partial crawl/resume; duplicate URL; prompt injection in page content.	Grounded correctness, citation precision/recall, abstention quality, freshness, ingestion success, latency, cost per query.
Task 2 — Inbox Agent	How-to; refund; cancellation; complaint; quote; project request; follow-up; ambiguous email; missing evidence; conflicting facts; hostile content; personal data; edited draft; rejected draft; streak reset; threshold reached; revocation; schema failure; retry exhaustion.	Intent accuracy, draft acceptance/edit rate, factuality, policy compliance, unsafe auto-send rate, cost/email, latency, reliability.
Task 3 — Meeting to Scope	Clear scope; vague search request; feature already exists; custom integration; conflicting requirements; missing acceptance criteria; long transcript; follow-up adds feature; removes feature; changes deadline; out-of-scope request; estimate uncertainty; no supporting product article; source-reference failure; proposal approval gate.	Requirement extraction F1 or rubric score, source traceability, feasibility accuracy, estimate error/range coverage, change-detection precision/recall, cost/run, latency, approval compliance.

9.2 Model comparison protocol

- Compare at least two model configurations, with Claude central to the solution as required by the brief. Compare models on the same cases and the same output schema.
- Record model/provider, model identifier, prompt version, date, input/output token counts where available, measured latency, retries, and estimated cost using the actual pricing applicable at test time.
- Use a repeatable rubric and, where possible, blind human scoring. Report averages and failure counts; include examples of regressions and not only aggregate scores.
- Do not fill the report with fabricated scores. If a comparison has not been run, mark it planned or pending and run it before submission.

9.3 Example evaluation row

Case ID	Input	Expected behaviour	Actual result	Quality	Latency	Cost	Pass/fail
KB-001	A question answered by one article	Correct answer with relevant source citation	Fill after execution	Score after review	Measure	Calculate	Pending

10. Safety, Reliability and Human Review

- Treat retrieved pages, emails, and transcripts as untrusted input. Instructions embedded inside them must not override system policy or tool permissions.
- Validate JSON/schema output before downstream use; retry only recoverable errors and cap retries.
- Set timeouts, request-size limits, rate limits, and bounded concurrency. Log errors with trace IDs but avoid logging secrets or unnecessary personal data.
- Require human approval for email sending and all Task 3 client-facing proposals until the defined autonomy/approval policy is satisfied.
- Keep API keys in environment variables or a local secrets file excluded from version control. Commit an .env.example with placeholder values only.
- Make important state transitions auditable and reversible: index version, prompt version, autonomy status, scope baseline, proposal approval, and rollback.
- Demonstrate at least one honest failure in the demo and explain how the system detects and contains it.

11. Repository Layout and One-Command Startup

Suggested repository structure:

ai-tech-consultant/

├── task1-knowledge-brain/ # Python API, ingestion, retrieval, evals

├── task2-inbox-agent/ # TypeScript inbox workflow and review UI

├── task3-meeting-to-scope/ # Python agents, estimates, proposal workflow

├── shared-contracts/ # OpenAPI schemas and sample payloads

├── data/

| ├── email_cases/ # 50+ authored emails

| └── meeting_transcripts/ # 3+ role-play transcripts

├── evals/

| ├── task1/ # 20+ cases

| ├── task2/ # 20+ cases

| └── task3/ # 20+ cases

├── scripts/ # Setup, seed, eval, demo helpers

├── README.md

├── DESIGN.md

├── EVALS.md

├── AI_LOG.md

├── .env.example

└── compose.yaml or start script

Choose a single root command such as:

docker compose up --build

or a single documented command such as:

./start.sh

The chosen command must be tested from a clean checkout. Include health checks, seed data, clear credential setup, and a shutdown/reset instruction.

12. Four-Day Execution Plan

Day	Priority work	End-of-day evidence
Day 1 — Foundation	Repository and startup; Task 1 sitemap ingestion; article normalization; metadata store; API contract; basic retrieval and cited answer.	Clean startup works; sample question returns source-backed answer; checkpoint and basic tests.
Day 2 — Inbox Agent	TypeScript email workflow; 50+ synthetic emails; classification; Task 1 integration;	End-to-end email demo; tests for edited draft, 20 approvals, reset, and revocation.

	draft UI/CLI; edit/approval audit; streak and revoke logic.	
Day 3 — Meeting to Scope	Three role-play transcripts; requirements schema; feasibility calls; estimate and proposal generation; follow-up comparison; human approval.	Scope/proposal generated with evidence; changes classified and approval gate tested.
Day 4 — Evaluation and delivery	20+ evaluations per task; compare two model configurations; fix highest-impact failures; finish docs; record demo; verify commits and live-review walkthrough.	Reproducible metrics; required documents; demo under 10 minutes; clean-clone startup and live-review notes.

13. Required Submission Documents

13.1 README.md — maximum 1 page

Include:

- One-sentence purpose and short description of the three tasks.
- Prerequisites, environment variables, and exact one-command startup.
- How to run each task, run tests/evaluations, and view the demo.
- Known limitations and links to DESIGN.md and EVALS.md.

13.2 DESIGN.md — 3 to 5 pages

Include:

- Problem in your own words; features included and why; features dropped and why.
- Architecture diagram showing how Tasks 1, 2, and 3 connect through Task 1's API.
- Agent responsibilities, data contracts, model choices, security, and approval boundaries.
- What breaks, known limitations, and next improvements.

13.3 EVALS.md — 2 to 3 pages

Include:

- At least 20 test cases per task, test methodology, and pass/fail rubric.
- At least two model/configuration comparisons, with measured quality, cost, and speed.
- Failure analysis, retries, safety tests, and limits of the results.

13.4 AI_LOG.md — maximum 1 page

Include:

- Five most useful prompts used with the selected AI coding tool.
- One example where the coding assistant's answer was rejected, with the reason.
- Retain the required end-to-end chat history for the single chosen coding tool.

13.5 Demo video — maximum 10 minutes

Time	Content
0:00–0:30	Show your face and introduce the problem and project.
0:30–2:45	Task 1: ask a question, show citations, then demonstrate an unknown answer or stale-source handling.
2:45–5:00	Task 2: process an email, show evidence and draft, edit/approve it, and explain the 20-approval safeguards.
5:00–7:45	Task 3: process a meeting, show requirements/estimates/proposal, then show follow-up changes and human approval.
7:45–9:15	Show one real failure case and how the system responds safely.
9:15–10:00	Show evaluation metrics, one-command startup, repository structure, and known limitations.

14. Evaluation Rubric and Final Readiness

Scoring part	Maximum	Full-score evidence
Problem thinking and feature ideas	15	Business value, thoughtful features, explicit trade-offs, and needs discovered independently.
Design: UI/UX, architecture, agents	20	Usable screens, clear responsibilities, architecture diagram matching the code, clean handoffs.
Code quality	15	Typed code, tests, one-command startup, sensible commits spread across the four days.

Prompt design and self-improvement	15	Versioned prompts, structured outputs, measurable and reversible learning from feedback.
Evals: quality, cost, speed, reliability, safety	20	20+ cases per task, at least two model configurations compared, measured cost/speed, retries, and approval controls.
Writing and demo	15	Clear short documents and a demo showing both success and failure.
Total	100	Fit requires 80+ overall and at least 60% of each scoring part.

The brief also specifies a 30-minute live review in which the student must explain the code and make a small change.

Hidden tests may include 10–30 unseen questions, emails, and transcripts. Be prepared to explain design choices, trace one request end-to-end, change a prompt or schema safely, and rerun the relevant tests.

15. Final Submission Checklist

Use this checklist before submitting:

- One repository with separate folders for all three tasks.
- Task 2 and Task 3 call Task 1 through its API; no copied Task 1 implementation.
- Only the permitted language per task; Claude is central; one AI coding tool used and its chats retained.
- Task 1 ingestion is polite, resumable, image-aware, and incrementally refreshed.
- Task 2 includes at least 50 realistic emails and tested 20-consecutive-approval safeguards.
- Task 3 includes at least three role-played transcripts, with a follow-up changing the original scope.
- Every task has at least 20 evaluation cases.
- At least two model/configuration choices are compared with measured quality, cost, and latency.
- README.md, DESIGN.md, EVALS.md, and AI_LOG.md meet the brief's length expectations.

- One-command startup tested from a clean checkout.
- Demo video is no longer than 10 minutes, includes face for the first 30 seconds, and includes a failure case.
- Secrets excluded, evidence traceable, human approval gates tested, and limitations stated honestly.
- Repository is ready for a 30-minute live review and hidden tests.

Appendix A. Suggested API Example

Endpoint

POST /v1/ask

Example request

```
{
  "question": "How do I configure this feature?",
  "top_k": 5
}
```

Example response shape (illustrative only)

```
{
  "status": "answered",
  "answer": "Use the settings described in the cited article...",
  "sources": [
    {
      "title": "Relevant help article",
      "url": "https://support.example.com/article",
      "snippet": "Supporting passage..."
    }
  ],
  "limitations": [],
  "request_id": "req_example"
```

}

Replace illustrative example values with actual source-backed values in the implementation. The sample URL and answer are placeholders, not real product documentation.

Appendix B. Suggested Risk Register

Risk	Impact	Mitigation
Help center unavailable or crawl blocked	Stale or incomplete knowledge	Backoff, checkpoints, last-known-good index, health status, and visible freshness timestamp.
Hallucinated answer or bad citation	Customer misinformation	Evidence requirement, citation validation, abstention, human escalation, and grounding evaluations.
Approval streak counts edited drafts	Unsafe automatic reply	Define immutable event types, unit tests, audit log, and explicit per-category eligibility.
Follow-up change is missed	Unpriced work or schedule slip	Compare structured requirements to approved baseline; test add/remove/modify/out-of-scope cases.
Estimates presented as certainty	Commercial risk	Show ranges, assumptions, dependencies, confidence, and human estimator review.
Model costs or latency exceed target	Poor business case	Measure per task, cap context, cache safe results, compare model tiers, and route only when supported by evaluations.

Source Note

This document is based on the user-provided “Business Labs — Assignment Prep Ref Material” and its attached capstone brief dated 14 October 2026. The assignment-specific requirements, deadlines, task languages, submission artifacts, scoring rubric, and 20-consecutive-approval rule are drawn from that source.

Architecture details, API examples, suggested schemas, and execution schedules are implementation recommendations to help operationalize the brief; validate them against your actual code and results.

Confidentiality: The supplied reference material is marked CONFIDENTIAL. Share the resulting document only with people authorized to receive the assignment material.